

Rapport d'Architecture Logicielle — Jeu des Schtroumpfs

Introduction

Ce rapport présente l'architecture logicielle du jeu de gestion de village Schtroumpf, développé dans le cadre de la SAÉ du BUT Informatique. Le projet implémente un jeu de stratégie au tour par tour où le joueur gère les ressources, les crises et les capacités des personnages pour assurer la survie et la prospérité du village.

L'architecture repose sur une **triade collaborative** entre trois piliers fondamentaux : l'état global du jeu (`Game`), la logique métier (`SmurfVillage`) et le pont vers l'interface utilisateur (`GameController`). Cette séparation garantit un **couplage faible**, une **source de vérité unique** pour chaque domaine de données, et une **extensibilité** maximale.

Le document détaille successivement l'architecture du moteur central, le système de personnages et de capacités, le moteur de ressources et de modificateurs, le système de crises et d'événements aléatoires, ainsi que le mécanisme de persistance des données.

1. Architecture du Moteur Central

1.1 Routeur Central : `AppController`

La classe `AppController` agit comme le **routeur de navigation central** de l'application. Elle possède une pile d'enregistrements `AppWindow`, chacun regroupant un `WindowType` et un `FxWindow<WindowSubController>` préchargé ou compilé dynamiquement. La conception basée sur une pile reflète un **automate à pile** typique et permet à l'application de prendre en charge à la fois les transitions modales (`push/pop`) et non modales (`replace`).

Actions de Navigation

L'énumération interne `NavigationAction` définit cinq actions possibles :

- **PUSH** – Place une nouvelle fenêtre au sommet de la pile, la rendant active. La fenêtre est soit récupérée depuis un cache préchargé (`preloadedWindows`), soit compilée à la volée via `compileLayout()`.
- **POP** – Retire la fenêtre la plus haute de la pile. Si la pile devient vide, l'application se ferme.
- **REPLACE** – Dépile la fenêtre courante puis pousse la fenêtre cible, remplaçant ainsi la vue la plus haute sans augmenter la profondeur de la pile.
- **STAY** – Aucune navigation n'a lieu ; la fenêtre courante reste inchangée.
- **EXIT** – Termine immédiatement l'application JavaFX.

La méthode `navigate(NavigationAction, WindowType)` implémente la logique de commutation et appelle `updateWindow()` pour définir la racine de la `Scene` partagée sur le

nouveau nœud racine de la fenêtre.

Préchargement des Fenêtres

Pour améliorer la réactivité, `AppController` précharge un sous-ensemble de fenêtres au moment de la construction (actuellement uniquement `SETTINGS_WINDOW`). La méthode `preloadWindows()` appelle `compileLayout()` pour chaque type souhaité et stocke le `FXMLWindow` résultant dans un `EnumMap<WindowType, FXMLWindow<WindowSubController>>`. Lorsqu'une action `PUSH` cible une fenêtre préchargée, l'instance mise en cache est utilisée directement, évitant ainsi le surcoût de l'analyse FXML à l'exécution.

Récupération du Contrôleur

La méthode générique `getWindowController(WindowType type)` itère sur la pile courante et retourne le contrôleur de la première fenêtre dont le type correspond au type demandé. Cela permet à n'importe quelle partie du système d'obtenir une référence à un contrôleur de fenêtre spécifique (par exemple, le `GameController`) sans couplage fort.

1.2 Modèle de Résultat de Navigation

La classe `Navigation` définit également un enregistrement `NavigationResult(NavigationAction action, WindowType target)`. Cet enregistrement est retourné par les méthodes `SubController.handle()` et est consommé par le `AppController` basé sur console pour décider de la prochaine étape de navigation. Le `AppController` basé sur l'interface graphique n'utilise pas cet enregistrement directement ; il s'appuie plutôt sur la méthode `navigate()` appelée depuis les contrôleurs de fenêtre.

1.3 État de l'Application : Classe Game

La classe `Game` est le **gestionnaire d'état central** pour la logique du jeu. Elle encapsule :

- L'instance `SmurfVillage`, qui contient les ressources, les membres du conseil, l'historique des événements et les compteurs de crises.
- Le numéro du tour courant (`currentRound`), allant de 1 à `MAX_ROUNDS` (12).
- La phase de jeu courante (`currentPhase`), une instance d'une classe implémentant `GamePhase`.
- L'état global du jeu (`gameState`), une énumération avec les valeurs `RUNNING`, `VICTORY` ou `DEFEAT`.

Initialisation

`startFirstMonth()` définit le tour à 1, initialise le village avec des quantités de ressources par défaut (3 unités de chaque `ResourceType`), sauvegarde l'instantané des ressources et définit la première phase à `ProductionPhase`. Cette méthode est appelée une fois lorsqu'une nouvelle partie commence.

Exécution et Avancement des Phases

La méthode `executePhaseLogic()` invoque le rappel `onEnter` de la phase courante, en passant un `GamePhaseContext` qui fournit l'accès au `Game`, au `SmurfVillage` et au numéro du tour courant. Après l'exécution de la logique de phase, `advance()` appelle `onExit` sur la phase courante puis obtient la phase suivante via `currentPhase.getNextPhase()`. Si la phase suivante est `null`, le mois est terminé et `handleMonthEnd()` est invoquée.

Logique de Fin de Mois

`handleMonthEnd()` effectue les étapes suivantes :

1. Vérifie si le village a été vaincu (trois crises ou plus). Si c'est le cas, l'état du jeu est défini à `DEFEAT`.
2. Incrémente le compteur de tours.
3. Vérifie la condition de victoire : si le tour dépasse `MAX_ROUNDS`, l'état du jeu est défini à `VICTORY`.
4. Sinon, appelle `village.prepareNextRound()` pour réinitialiser les compteurs par tour et définit la phase suivante à une nouvelle `ProductionPhase`.

Support de Sauvegarde/Chargement

La méthode `loadSave(GameSave save)` restaure l'état du moteur à partir d'un enregistrement `GameSave.EngineState`, qui contient le tour courant, l'état du jeu et le type de phase courante. Cela permet de reprendre la partie à partir d'un état précédemment sauvegardé.

API Publique

La classe `Game` expose plusieurs accesseurs pour la couche de contrôle : `getVillage()`, `getCurrentRound()`, `getCurrentPhase()` et `getGameState()`. Ces méthodes permettent aux contrôleurs de l'interface graphique d'interroger l'état du jeu et de mettre à jour l'interface utilisateur en conséquence.

1.4 Orchestration du Moteur de Jeu

L'architecture du moteur repose sur une **triade collaborative** où chaque composant possède une responsabilité unique et bien délimitée. Cette séparation garantit un **couplage faible** entre la logique métier, l'état global et la couche de présentation, tout en maintenant une **source de vérité unique** pour chaque domaine de données.

Game — L'État Global et le Coordinateur

La classe `Game` agit comme le **point d'entrée unique** pour l'état global de la partie. Elle ne contient **aucune logique métier complexe** ; son rôle est exclusivement celui d'un **orchestrateur** qui :

- Maintient le **compteur de rounds** (`currentRound`) et l'**état de la partie** (`gameState` : `RUNNING`, `VICTORY`, `DEFEAT`).
- Possède une référence unique vers l'instance de `SmurfVillage`, qui constitue le modèle

de données du village.

- Gère la **machine à états des phases** : `executePhaseLogic()` invoque le callback `onEnter` de la phase courante, puis `advance()` appelle `onExit` et détermine la phase suivante via `getNextPhase()`.
- Détecte les **conditions de fin de partie** dans `handleMonthEnd()` (défaite si trois crises ou plus, victoire après 12 rounds).

Principe de délégation : `Game` ne calcule jamais directement l'impact d'une action. Il transmet un `GamePhaseContext` (contenant `Game`, `SmurfVillage` et le round courant) à chaque phase, qui délègue à son tour à `SmurfVillage` toute manipulation de données.

SmurfVillage — Le Cœur de la Logique Métier

La classe `SmurfVillage` constitue le **noyau décisionnel** du jeu. Elle centralise l'ensemble des règles métier et expose une API publique que `Game` et les phases consultent sans jamais exposer ses structures internes.

Responsabilités clés :

- **Gestion des ressources** : via `ResourceManager`, elle contrôle les quantités, les plafonds (`MAX_QUANTITY = 10`), et les différences entre rounds (`getResourcesDiff()`).
- **Moteur de modificateurs** : le `VillageModifierContext` (`modifiers`) accumule les effets temporaires et persistants. Les méthodes comme `getDynamicEffectDelta()`, `getDynamicMaxEnergy()` ou `getProductionRate()` appliquent ces modificateurs en temps réel, garantissant que toute requête reflète l'état actuel du jeu.
- **Gestion des crises** : `applyCrises()` filtre les crises entrantes via le bouclier (`CRISIS_SHIELD_COUNT`), puis `applyActiveCrisesModifiers()` applique leurs effets au début de chaque round.
- **Système de callbacks** : via `registerCallback()` et `runCallback()`, le village notifie la couche vue des changements de modificateurs sans connaître son existence — un exemple concret de **couplage faible**.

Single Source of Truth : Toute donnée dynamique (ressources, énergie des schtroumpfs, modificateurs actifs) est stockée et calculée exclusivement dans `SmurfVillage`. Ni `Game` ni `GameController` ne dupliquent ces informations.

GameController — Le Pont entre le Modèle et la Vue

La classe `GameController` agit comme un **médiateur** entre le moteur de jeu et l'interface utilisateur JavaFX. Elle ne contient **aucune logique métier** ; son rôle est de :

- **Observer l'état** : elle possède une instance de `Game` et interroge `SmurfVillage` via des getters pour obtenir les ressources, les crises, les modificateurs et les membres du conseil.
- **Déclencher les transitions** : la méthode `advanceTurn()` appelle `game.advance()`, puis vérifie l'état résultant (`VICTORY`, `DEFEAT`, `RUNNING`) pour charger la vue appropriée.

- **Mettre à jour l'interface** : `updateHudResources()`, `updateHudCrisis()` et `updateHudTotalModifiers()` lisent les données depuis `SmurfVillage` et rafraîchissent les widgets JavaFX correspondants.
- **Réagir aux callbacks** : via `registerVillageCallbacks()`, elle s'abonne aux notifications de `SmurfVillage` (notamment `MODIFIERS_UPDATED`) pour mettre à jour la vue sans polling actif.

Injection de dépendance : `GameController` reçoit le `AppController` via `setRouter()` et transmet ce dernier aux sous-contrôleurs de phase via `FxmlUtils.loadFxmlAndPassController()`, permettant une navigation modulaire sans couplage direct.

Le Système de Phases — Machine à États du Jeu

Le déroulement d'un tour de jeu est structuré autour d'une **machine à états séquentielle** implémentée par l'interface `GamePhase` et ses concrétisations. Chaque phase représente une étape logique du mois en cours, avec une entrée (`onEnter`), une sortie (`onExit`) et une transition vers la phase suivante (`getNextPhase`).

Contrat de l'Interface `GamePhase`

L'interface `GamePhase` définit trois méthodes fondamentales :

- **`onEnter(GamePhaseContext ctx)`** : Invoquée automatiquement par `Game.executePhaseLogic()` lorsque la phase devient active. C'est ici que la logique métier de la phase est exécutée (production de ressources, déclenchement d'événements, etc.).
- **`onExit(GamePhaseContext ctx)`** : Invoquée par `Game.advance()` avant de passer à la phase suivante. Permet le nettoyage ou la journalisation.
- **`getNextPhase()`** : Retourne l'instance de la phase suivante, ou `null` pour signaler la fin du mois.

La méthode statique `fromType(GamePhaseType type)` agit comme une **fabrique** qui convertit un type énuméré en instance concrète, permettant la désérialisation depuis une sauvegarde.

Séquence des Phases d'un Mois

Chaque mois (round) suit un ordre immuable de cinq phases :

```
ProductionPhase → EventPhase → CouncilPhase → ConsumptionPhase → CrisisPhase → (fin de mois)
```

1. **ProductionPhase** : Génère des ressources passives et recharge l'énergie des schtroumpfs. Le village dispose d'un taux de production (`getProductionRate()`) qui détermine le nombre d'unités de ressources à distribuer. Pour chaque unité, une ressource éligible est sélectionnée aléatoirement parmi celles qui ne sont pas saturées. Les ressources bloquées par des modificateurs sont exclues. En fin de phase, chaque membre du conseil récupère de l'énergie via `village.rechargeSmurfEnergy()`. La transition suivante est `EventPhase`.

2. **EventPhase** : Déclenche un événement aléatoire qui impacte le village.
`RandomEventGenerator.nextEvent(currentRound)` sélectionne un événement parmi les types définis, avec une fréquence de base modulée par le round. L'événement est exécuté via `event.trigger(village)`, qui retourne une liste d'effets sur les ressources. Ces effets sont appliqués via `village.applyEffects()`, et l'événement est enregistré dans l'historique du village. La transition suivante est `CouncilPhase`.
3. **CouncilPhase** : Permet au joueur d'utiliser les capacités spéciales des membres du conseil. Cette phase est **interactive** : elle attend que le joueur sélectionne un schtroumpf et une capacité via l'interface graphique. Le contrôleur de phase (`PhaseSubController`) appelle `village.executeCouncilMemberAbility()` qui vérifie les préconditions (énergie suffisante, limite d'actions non atteinte), consomme l'énergie, applique les effets, et incrémente le compteur d'actions du tour. La limite d'actions est dynamique (`getDynamicMaxAbilitiesPerTurn()`), modulée par les modificateurs actifs. La transition suivante est `ConsumptionPhase`.
4. **ConsumptionPhase** : Applique les coûts de maintenance et les règles de consommation du village. Un ensemble de `ConsumptionRule` est évalué séquentiellement. Chaque règle (alimentation, surpopulation, chauffage hivernal, décaissement des infrastructures) examine l'état actuel du village et applique des pénalités si les ressources sont insuffisantes. Par exemple, `FoodRule` consomme des baies proportionnellement à la population, et déclenche une crise si le stock est épuisé. Les résultats sont agrégés dans un `ConsumptionReport`. La transition suivante est `CrisisPhase`.
5. **CrisisPhase** : Évalue et applique les crises déclenchées par les phases précédentes. Pour chaque type de crise défini dans `CrisisType`, la méthode `shouldTrigger(village)` est appelée. Si une condition de déclenchement est remplie, une instance de `Crisis` est créée et ajoutée à la liste. `village.applyCrises()` filtre ensuite ces crises via le bouclier (`CRISIS_SHIELD_COUNT`) et les active. Les modificateurs des crises sont appliqués au début du round suivant via `applyActiveCrisesModifiers()`. La transition suivante est `null`, signalant la fin du mois.

Gestion de la Fin de Mois

Lorsque `getNextPhase()` retourne `null`, `Game.handleMonthEnd()` est invoqué. Cette méthode :

1. Vérifie la condition de défaite (`village.isDefeated()`, soit 3 crises ou plus).
2. Incrémente le compteur de rounds.
3. Vérifie la condition de victoire (`currentRound > MAX_ROUNDS`, soit 12 mois).
4. Appelle `village.prepareNextRound()` pour sauvegarder l'état des ressources, réinitialiser le compteur d'actions, et appliquer les modificateurs des crises actives.
5. Initialise une nouvelle `ProductionPhase` pour le mois suivant.

Intégration avec la Vue

Chaque phase possède un fichier FXML associé, défini dans l'énumération `GamePhaseType` via `getFXMLFile()`. Lorsque `GameController.syncPhaseView()` détecte un changement de phase, elle charge dynamiquement la vue correspondante dans le panneau central (`centerContainer`) via `FxmlUtils.loadFXMLAndPassController()`. Le contrôleur de phase (`PhaseSubController`) reçoit une référence au `GameController` maître, lui permettant de déclencher `advanceTurn()` une fois l'interaction utilisateur terminée.

Avantage architectural : Cette séparation permet d'ajouter ou de modifier une phase sans impacter les autres. Chaque phase est une unité autonome avec sa propre logique, sa propre vue FXML, et son propre contrôleur. La machine à états garantit que l'ordre d'exécution reste cohérent et prévisible.

2. Système de Personnages et de Capacités

2.1 Architecture contractuelle via `SmurfCharacter`

L'interface `SmurfCharacter` constitue le pivot architectural du système de personnages. Elle définit un contrat strict que chaque type de Schtroumpf doit respecter, garantissant ainsi une polymorphie totale au sein du moteur de jeu.

Objectif : Permettre au moteur de jeu (`SmurfVillage`, phases de jeu) d'interagir avec n'importe quel personnage sans connaître son type concret, conformément au principe de substitution de Liskov.

Structure contractuelle :

- Gestion de l'énergie : `getEnergy()`, `setEnergy()`, `updateEnergy()`, `getBaseMaxEnergy()`
- Gestion des attributs : `getAttribute()`, `updateAttribute()`
- Exposition des capacités : `getAbilities()`
- Méthodes de commodité : `hasEnoughEnergy()`, `hasRequiredResources()`, `canExecute()`
- Factory statique : `fromType(SmurfType type)`

Justification du choix de l'interface : L'interface a été préférée à une classe abstraite pour plusieurs raisons :

1. **Principe Ouvert/Fermé :** L'ajout d'un nouveau Schtroumpf se limite à l'implémentation de l'interface sans modifier le moteur de jeu. La factory `fromType()` est le seul point d'extension à mettre à jour.
2. **Absence d'état partagé :** Chaque personnage gère son propre état (énergie, attributs) via des `HashMap` internes, rendant inutile un héritage d'état.
3. **Flexibilité maximale :** L'interface permet à chaque implémentation de définir sa propre stratégie de comportement sans contrainte d'héritage.

2.2 Encapsulation des capacités via `CharacterAbility`

`CharacterAbility` est implémentée comme un *record* Java, ce qui lui confère immutabilité et transparence structurelle. Elle encapsule l'ensemble des métadonnées d'une capacité.

Structure d'une capacité :

- `name` : nom d'affichage
- `description` : description textuelle
- `energyCost` : coût en énergie
- `requiredResources` : ressources nécessaires (`List<ResourceSnapshot>`)
- `primaryEffects` : effets principaux (pour l'UI uniquement)
- `actionLogic` : **fonction lambda** de type `Function<SmurfVillage, AbilityResult>`

Utilisation des lambdas comme stratégie d'exécution : Le champ `actionLogic` est une `Function<SmurfVillage, AbilityResult>`, ce qui constitue l'élément clé du design. Cette approche présente plusieurs avantages :

1. **Séparation définition/implémentation** : La définition de la capacité (nom, coût, prérequis) est déclarative dans le record, tandis que la logique d'exécution est encapsulée dans une méthode privée de la classe personnage.
2. **Pattern Stratégie** : Chaque capacité est une stratégie autonome, injectée dans le record via une référence de méthode (`this::executeGatherBerries`).
3. **Testabilité** : La logique d'exécution peut être testée indépendamment de la structure de la capacité.

Résultat d'exécution : Le type `AbilityResult` (record interne) encapsule :

- `type` : `SUCCESS`, `FAILURE` ou `NEUTRAL` (pour l'UI)
- `message` : description textuelle du résultat
- `effectsToApply` : liste d'effets à appliquer au village

2.3 Gestion des attributs et état

Chaque personnage gère son état interne via deux mécanismes distincts :

Gestion de l'énergie :

- Valeur initiale commune : 10
- Méthode `updateEnergy()` utilisant `Math.clamp()` pour borner entre 0 et `getDynamicMaxEnergy()` (calculé dynamiquement via les modificateurs du village)
- `getBaseMaxEnergy()` : valeur par défaut de 10, surchargeable

Gestion des attributs personnalisés :

- Stockage via `Map<CharacterAttribute, Integer>` (implémentation `HashMap`)
- Initialisation spécifique dans chaque constructeur

Cette approche permet une personnalisation totale du comportement : les attributs influencent directement les probabilités de succès des capacités (ex: `GrandSmurf.executeCheckSpellBook` utilise `WISDOM` pour moduler la chance de réussite).

2.4 Différenciation concrète des personnages

L'interface `SmurfCharacter` permet d'implémenter des stratégies radicalement différentes tout en respectant le même contrat.

BrainySmurf (orientation érudition) :

- Attribut clé : `WISDOM` = 2 (le plus élevé)
- Capacités orientées savoir : traduction de formules, étude de parchemins, écriture d'histoire
- Interaction avec les modificateurs : `executeStudyScroll()` applique un `ModifierEffect` de type `PRODUCTION_DELTA`
- Utilisation de `WeightedOutcomeSelector` pour les issues aléatoires (traduction de formule)

GrouchySmurf (orientation survie/défense) :

- Capacités défensives : surveillance des alentours (+1 Défense), dénonciation, prévention d'attaque
- Capacité unique : `executePreventAttack()` applique un modificateur persistant `CRISIS_SHIELD_COUNT`, démontrant une interaction avancée avec le système de crise
- Pas d'utilisation de `WeightedOutcomeSelector` : résultats déterministes

GrandSmurf (orientation village/leader) :

- Attribut clé : `WISDOM` = 0 (évolutif via succès)
- Capacités sociales : consultation du grimoire, réunion, négociation avec les animaux
- Utilisation intensive de `WeightedOutcomeSelector` avec probabilités dynamiques basées sur `WISDOM`
- Pattern d'apprentissage : `executeCheckSpellBook()` incrémente `WISDOM` en cas de succès, créant une boucle de progression

2.5 Place des utilitaires dans l'architecture

Le `WeightedOutcomeSelector` est un composant utilitaire optionnel, non intégré au cœur de l'architecture des capacités. Il intervient uniquement lorsque la logique métier nécessite un tirage aléatoire pondéré entre plusieurs issues possibles.

Fonctionnement :

- Agrège des `OutcomeChoice` (record contenant poids, type de résultat, message, effets, callback optionnel)
- Calcule les probabilités normalisées par catégorie (succès/échec/neutre)
- Applique les modificateurs de chance du village (`SUCCESS_CHANCE_BONUS`)
- Sélectionne une issue via tirage aléatoire

Caractère optionnel : Ce composant n'est pas un prérequis de l'architecture. Les capacités déterministes (ex: `GrouchySmurf.executeWatchSurroundings`) l'ignorent complètement, prouvant la flexibilité du design.

2.6 Conclusion architecturale

Le système de personnages implémente un **Pattern Stratégie** à deux niveaux :

1. **Niveau macro** : Chaque `SmurfCharacter` est une stratégie globale de comportement
2. **Niveau micro** : Chaque `CharacterAbility` est une stratégie d'action individuelle, paramétrée par son `actionLogic` lambda

Cette architecture respecte le principe **Ouvert/Fermé** : le système est ouvert à l'extension (nouveaux personnages, nouvelles capacités) mais fermé à la modification (le moteur de jeu n'a pas besoin d'être modifié). L'utilisation de records immutables et de lambdas garantit une séparation claire entre la définition déclarative et l'implémentation procédurale, facilitant la maintenance et l'évolution du projet.

3. Moteur de Ressources et de Modificateurs

3.1 Architecture du `VillageModifierContext`

Le `VillageModifierContext` constitue le cœur du système de règles dynamiques du village. Son architecture repose sur un modèle hybride combinant deux structures de données aux cycles de vie distincts :

- **Une `Map<GameModifierType, Object>` pour les effets persistants** : ces modificateurs s'accumulent par addition via la méthode `accumulatePersistentEffect()`. Ils représentent des altérations durables du jeu, comme les boucliers anti-crisés (`CRISIS_SHIELD_COUNT`) ou les bonus d'équipement. Leur valeur est calculée par combinaison additive : chaque appel à `accumulatePersistentEffect` invoque la fonction `combine()` propre à chaque `GameModifierType`, garantissant la cohérence typée des opérations.
- **Une `List<ModifierEffect>` pour les effets temporaires** : chaque effet est encapsulé dans un objet `ModifierEffect` qui porte sa durée résiduelle en tours (`remainingRounds`). Cette liste permet de gérer des effets multiples de même type sans écrasement — par exemple, deux crises actives appliquant simultanément un malus de production.

La méthode générique `get(GameModifierType type)` centralise le calcul de la valeur finale en combinant la valeur par défaut, les modificateurs persistants et les modificateurs temporaires. Ce mécanisme exploite le polymorphisme des `GameModifierType` : chaque énumération définit sa propre fonction de combinaison (`addInt`, `addDouble`, `addBool`) et sa valeur par défaut. Le système est ainsi extensible sans modification du contexte — l'ajout d'un nouveau type de modificateur ne nécessite que la déclaration d'une nouvelle constante dans l'énumération.

3.2 Système de Hooks : le Moteur de Règles

`SmurfVillage` expose une série de méthodes qui agissent comme des points d'interception (*hooks*) entre les mécaniques de base du jeu et le contexte de modificateurs. Ces hooks constituent le véritable "moteur de règles" du jeu, permettant aux crises, aux capacités des Schtroumpfs et aux événements d'altérer dynamiquement le comportement du village.

Interception des jets de chance

La méthode `rollChance(double baseChance)` remplace l'appel direct à `GameRandomness.rollChance()` en appliquant le modificateur `SUCCESS_CHANCE_BONUS`. Ce hook permet à une crise comme `RevoltCrisis` d'appliquer un malus de -25% à toutes les chances de succès des capacités, sans que les capacités elles-mêmes aient à connaître l'existence des crises.

Interception des valeurs limites

Plusieurs hooks ajustent les capacités maximales des entités du jeu :

- **`getDynamicMaxEnergy(SmurfCharacter)`** : calcule l'énergie maximale d'un Schtroumpf en fonction du modificateur `MAX_ENERGY_DELTA`. Une `FamineCrisis` réduit cette valeur de 2, limitant le nombre d'actions possibles.
- **`getDynamicMaxAbilitiesPerTurn()`** : détermine le nombre d'actions autorisées par tour. La `BankruptcyCrisis` applique un `ABILITIES_PER_TURN_DELTA` de -1, réduisant la capacité d'action du conseil.
- **`getProductionRate()`** : ajuste le nombre de ressources produites pendant la phase de production. La `StoneAgeCrisis` applique un `PRODUCTION_DELTA` de -1.

Interception des effets de ressources

La méthode `getDynamicEffectDelta(ResourceEffect)` applique le modificateur d'efficacité (`EFFICIENCY_DELTA`) à chaque effet de ressource. Cette logique garantit qu'un effet positif ne peut être réduit à zéro (plancher à 1), préservant ainsi une progression minimale même en période de crise.

3.3 Gestion du Cycle de Vie des Modificateurs

L'intégrité des données au fil des tours est assurée par deux mécanismes complémentaires :

`tickRounds()`

Invoquée à chaque début de tour via `applyActiveCrisesModifiers()`, cette méthode

décrémente le compteur de tours de chaque `ModifierEffect` temporaire. Les effets arrivés à expiration sont automatiquement retirés de la liste active. Un mécanisme de *lazy start* (`started` flag) diffère le début du décompte au tour suivant l'application, évitant qu'un effet ne expire prématurément dans le tour où il a été appliqué.

clearCrisisEffects()

Cette méthode filtre les effets temporaires pour ne conserver que ceux qui ne proviennent pas d'une crise (`isCrisis() == false`). Elle est utilisée lors de la résolution des crises pour nettoyer les modificateurs des crises précédentes avant d'appliquer ceux des nouvelles crises. Cette isolation garantit qu'une crise ne laisse pas de traces persistantes après sa résolution, tandis que les buffs permanents (acquis via des capacités ou des événements) demeurent actifs.

3.4 Interaction avec le Système de Ressources

ResourceManager

Le `ResourceManager` encapsule une `ResourceMap` (extension de `EnumMap<ResourceType, Integer>`) et assure le respect de la contrainte de capacité maximale (`MAX_QUANTITY = 10`). Il expose trois opérations fondamentales :

- `set(ResourceType, int)` : affecte une quantité, avec clamps entre 0 et `MAX_QUANTITY`
- `add(ResourceType, int)` : incrémente ou décrémente, avec clamps automatique
- `get(ResourceType)` : retourne la quantité courante

Application dynamique des effets

La méthode `applyEffects(List<ResourceEffect>)` dans `SmurfVillage` orchestre l'interaction entre les effets bruts et le moteur de modificateurs. Chaque effet de capacité ou d'événement passe ainsi par le filtre du modificateur d'efficacité avant d'affecter les ressources. Une capacité produisant normalement +3 baies ne produira que +2 si une `EpidemicCrisis` est active (efficacité -1).

3.5 Collaboration et Modularité

L'architecture décrite permet une séparation nette des préoccupations :

- **Les capacités des Schtroumpfs** (`CharacterAbility`) définissent des effets bruts (ex: "+3 baies") sans connaître l'état des crises.
- **Les crises** (`Crisis`) définissent des modificateurs contextuels (ex: "-1 efficacité") sans connaître les capacités individuelles.
- **Le VillageModifierContext** sert de médiateur, combinant les contributions de toutes les sources pour produire les valeurs effectives.
- **Les hooks de SmurfVillage** interceptent chaque point d'entrée du jeu pour injecter les modifications.

Cette conception, inspirée du patron *Decorator* appliqué à l'échelle du système de règles, permet

d'ajouter de nouvelles crises, capacités ou événements sans modifier les classes existantes — seule la déclaration de nouveaux `GameModifierType` et l'implémentation des hooks correspondants sont nécessaires.

4. Système de Crise et d'Événements Aléatoires

4.1 Architecture Générale

Le jeu intègre deux systèmes distincts mais complémentaires de perturbations : les **crises** et les **événements aléatoires**. Ces deux mécanismes partagent une philosophie commune : ils modifient l'état du village via des effets temporaires ou immédiats, mais diffèrent dans leur déclenchement et leur cycle de vie.

4.2 Système de Crise

Déclenchement d'une Crise

Une crise est déclenchée lorsque la ressource causale (*cause*) atteint zéro. Cette logique est implémentée dans la méthode `shouldTrigger()` de l'énumération `CrisisType`. Chaque type de crise est associé à une ressource spécifique :

- FAMINE → BERRIES
- EPIDEMIC → SARSAPARILLA
- REVOLT → MORAL
- MASSIVE_ATTACK → DEFENSE
- DARK_AGES → KNOWLEDGE
- BANKRUPTCY → GOLD
- STONE_AGE → TOOLS

Cycle de Vie d'une Crise

Le cycle de vie d'une crise suit un modèle en trois phases :

1. **Détection** : À chaque tour, le système vérifie si une ressource est épuisée. Si c'est le cas, la crise correspondante est activée.
2. **Application des effets immédiats** : Certaines crises appliquent des effets instantanés via la méthode `applyImmediateEffects()`. Par exemple, `MassiveAttackCrisis` réduit immédiatement les outils et l'or du village.
3. **Application des modificateurs temporaires** : Chaque crise retourne une liste de `ModifierEffect` via `getModifierEffects()`. Ces effets sont créés avec la méthode statique `crisisModifierEffect()` qui les marque comme étant de type crise et leur attribue une durée d'un tour complet.

Types d'Effets de Crise

Les crises appliquent des modificateurs variés sur les mécaniques du jeu :

Crise	Modificateur	Valeur	Impact
Famine	MAX_ENERGY_DELTA	-2	Réduit l'énergie maximale des Schtroumpfs
Épidémie	EFFICIENCY_DELTA	-1	Réduit l'efficacité des actions
Révolte	SUCCESS_CHANCE_BONUS	-0.25	Réduit les chances de succès de 25%
Attaque Massive	MAX_ENERGY_DELTA	-3	Fatigue extrême des Schtroumpfs
Âges Sombres	PASSIVE_FOOD_PRODUCTION_BLOCKED	true	Bloque la production passive de nourriture
Faillite	ABILITIES_PER_TURN_DELTA	-1	Réduit le nombre d'actions disponibles
Âge de Pierre	PRODUCTION_DELTA	-1	Réduit la production globale

Gestion des Modificateurs

Les modificateurs de crise sont stockés dans `VillageModifierContext` sous forme d'effets temporaires. La classe `ModifierEffect` gère leur cycle de vie avec un compteur de tours restants (`remainingRounds`) et un drapeau `started` qui retarde le début du décompte jusqu'au prochain tour complet.

4.3 Système d'Événements Aléatoires

Génération des Événements

La classe `RandomEventGenerator` implémente un système de sélection pondérée basé sur la fréquence de chaque événement. La somme des fréquences de tous les événements constitue l'espace de tirage. Un nombre aléatoire est généré dans cet intervalle, puis on soustrait itérativement la fréquence de chaque événement jusqu'à obtenir une valeur négative ou nulle. L'événement correspondant est alors sélectionné.

Fréquence Dynamique

Chaque type d'événement possède une fréquence de base et, optionnellement, un modificateur de fréquence qui évolue avec le numéro du tour. Par exemple, `GargamelAttack` devient plus fréquent au fil du jeu, avec un modificateur qui augmente de 10 points par tour à partir du cinquième tour.

Catalogue des Événements

Événement	Fréquence de Base	Effets
GARGAMEL_ATTACK	35 (variable)	Perte de défense (-3) et moral (-2)
MAGIC_BERRIES	15	Gain de baies (+2) et salsepareille (+2)
FRIENDLY_VILLAGE	15	Gain d'or (+2), moral conditionnel (+2 si Smurfette a >3 énergie)
SARSAPARILLA_STORM	25	Perte d'outils (-3), gain de connaissance conditionnel (+1 si connaissance ≥ 3)
SMURF_PARTY	20	Perte de baies (-2), gain de moral conditionnel (+3 si baies ≥ 2)
FOREST_CURSE	30	Perte de connaissance (-3)

Exécution des Événements

Tous les événements implémentent l'interface `GameEvent` qui définit la méthode `trigger()`. Cette méthode retourne une liste d'effets de ressources (`ResourceEffect`) qui sont ensuite appliqués au village via `SmurfVillage.applyEffects()`. Le système utilise le patron **Factory Method** via `GameEvent.fromType()` et `Crisis.fromType()` pour instancier les objets concrets à partir de leur type énuméré.

4.4 Interaction avec les Règles du Jeu

Règles de Consommation

L'interface `ConsumptionRule` (définie comme `@FunctionalInterface`) permet de définir des règles de consommation de ressources qui s'appliquent pendant la phase de consommation. Ces règles interagissent avec le système de crise de deux manières :

- **Vérification des ressources** : Les règles de consommation peuvent déclencher des crises si elles réduisent une ressource à zéro.
- **Application des modificateurs** : Les effets de crise modifient les paramètres de consommation (ex : `PRODUCTION_DELTA` réduit la production).

Impact sur les Statistiques du Village

Les crises et événements affectent les statistiques du village via le système de modificateurs de `VillageModifierContext`. Ce système utilise un mécanisme d'**accumulation** où chaque modificateur est combiné avec la valeur existante via la méthode `combine()` du `GameModifierType`.

4.5 Considérations Architecturales

Séparation des Préoccupations

Le système sépare clairement :

- **La détection** (dans `CrisisType.shouldTrigger()`)
- **La définition des effets** (dans les classes internes de `Crises`)
- **L'application** (dans `SmurfVillage` et `VillageModifieurContext`)

Extensibilité

L'ajout d'une nouvelle crise ou d'un nouvel événement nécessite :

1. Ajouter une constante dans l'énumération correspondante
2. Implémenter l'interface (`Crisis` ou `GameEvent`)
3. Ajouter le cas dans la méthode `fromType()` de l'interface

Cette architecture respecte le principe **Open/Closed** : le système est ouvert à l'extension mais fermé à la modification.

5. Système de Sauvegarde (Game Saving)

5.1 Architecture générale

Le système de sauvegarde repose sur deux classes principales : `GameSaveManager` et `GameSave`. La première agit comme un orchestrateur de la sérialisation/désérialisation, tandis que la seconde définit la structure des données persistées sous forme de *Data Transfer Objects* (DTO).

5.2 Utilisation de Jackson pour la sérialisation JSON

La bibliothèque Jackson est utilisée pour convertir l'état du jeu en fichiers JSON. Le `ObjectMapper` est configuré avec l'option `INDENT_OUTPUT` pour produire des fichiers lisibles. La persistance s'effectue via deux méthodes principales :

- **`saveFile(String fileName, GameSave data)`** : écrit le DTO dans un fichier situé dans le répertoire `saves/`. Le chemin est construit selon le format `save_<nom>.json`. Jackson gère automatiquement la conversion des records Java en JSON grâce à ses annotations implicites sur les composants des records.
- **`loadFile(String fileName)`** : lit un fichier JSON et le désérialise en une instance de `GameSave`. Jackson utilise le constructeur canonique des records pour reconstruire les objets.

5.3 Pattern DTO avec le record GameSave

Le record `GameSave` implémente le pattern *Data Transfer Object* : il sert de conteneur purement structurel, sans logique métier, pour transporter l'état du jeu entre la mémoire et le disque. Sa

structure hiérarchique reflète la composition du modèle :

```
GameSave
├── EngineState (round, état, phase courante)
├── VillageState
│   ├── ressources courantes et précédentes (ResourceMap)
│   ├── membres du conseil (CouncilMemberState)
│   ├── historique des événements (EventHistory)
│   ├── crises actives (CrisisState)
│   └── modificateurs (VillageModifierCtxState)
```

Chaque sous-record (EngineState, VillageState, CouncilMemberState, CrisisState, VillageModifierCtxState, TemporaryModifierState) encapsule un sous-ensemble cohérent de l'état. Cette décomposition permet une sérialisation granulaire et facilite l'évolution du format de sauvegarde.

5.4 Hydratation du VillageModifierContext

Le VillageModifierContext est reconstruit à partir de l'état sauvegardé via un constructeur dédié qui accepte un VillageModifierCtxState. Ce constructeur itère sur les deux catégories de modificateurs :

1. **Modificateurs persistants** (persistentModifiers) : stockés dans une Map<GameModifierType, Object>, ils sont directement recopiés dans la map interne du contexte.
2. **Modificateurs temporaires** (temporaryModifiers) : chaque TemporaryModifierState est converti en une instance de ModifierEffect en conservant le type, la valeur, la durée restante et le drapeau isCrisis. Le champ started est réinitialisé à false par le constructeur de ModifierEffect, ce qui garantit que le compte à rebours des tours ne commence qu'à l'entrée du prochain round.

5.5 Flux de sérialisation complet

La méthode serializeGame(Game game) orchestre la conversion :

1. **État du moteur** : serialEngine() extrait le round courant, l'état du jeu (VICTORY, DEFEAT, RUNNING) et le type de phase active.
2. **État du village** : serializeVillage() collecte les ressources (converties de ResourceSnapshot en ResourceMap via resourcesSnapToMap), les membres du conseil (type et énergie), l'historique des événements, les crises actives (uniquement leur type) et l'état des modificateurs via getModifiersView().serialize().

La désérialisation suit le chemin inverse : deserializeGame() crée une nouvelle instance de Game, puis appelle game.loadSave(save) et village.loadSave(save.villageState()) pour hydrater chaque composant à partir des DTO.

5.6 Gestion des fichiers de sauvegarde

Le GameSaveManager expose getSaveNames() qui parcourt le répertoire saves/ à la

recherche de fichiers correspondant au motif `save_*.json`. Cette méthode extrait le nom de la sauvegarde en supprimant le préfixe `save_` et le suffixe `.json`, permettant ainsi une interface utilisateur pour charger une partie existante.

Conclusion

L'architecture logicielle du jeu des Schtroumpfs illustre une application rigoureuse des principes de conception orientée objet. La **triade collaborative** entre `Game`, `SmurfVillage` et `GameController` garantit un couplage faible et une séparation claire des préoccupations. Le système de phases implémente une machine à états séquentielle qui structure le déroulement du jeu de manière prévisible et extensible.

Le **Pattern Stratégie** à deux niveaux (personnages et capacités) permet une grande flexibilité d'évolution, tandis que le moteur de modificateurs, inspiré du patron *Decorator*, offre un mécanisme puissant pour altérer dynamiquement les règles du jeu sans modifier les classes existantes. Les systèmes de crises et d'événements aléatoires respectent le principe **Ouvert/Fermé**, facilitant l'ajout de nouveaux contenus.

Enfin, le système de sauvegarde basé sur Jackson et le pattern DTO assure une persistance fiable et évolutive de l'état du jeu. L'ensemble de ces choix architecturaux démontre une maîtrise des concepts avancés de génie logiciel et constitue une base solide pour l'évolution future du projet.